UNIVERSITÉ SORBONNE PARIS NORD

Sup Galilée ingénieurs
L'ÉCOLE D'INGÉNIEURS DE L'UNIVERSITÉ SORBONNE PARIS NORD

Compte rendu TP OS temps réel

Noms : Dahmoun, Sinacer
Prénoms : Sarah, Tedj El Moulk

ing2 instrumentation

Introduction :

Ce TP a pour objectif de découvrir et mettre en œuvre un système de contrôle HVAC (chauffage, ventilation, climatisation) en utilisant un système temps réel (FreeRTOS). Nous avons d'abord testé différentes fonctions sur Arduino, comme la gestion du temps avec et sans blocage, ainsi que la création de tâches avec FreeRTOS. Ensuite, nous avons reproduit ceci sur STM32Cube IDE avec une carte STM32. Initialement prévu avec une communication Wi-Fi/MQTT, le projet a été simplifié : les capteurs et actionneurs ont été simulés avec des composants électroniques (LEDs, résistances, potentiomètres…). Ce TP nous a permis de comprendre les bases de la programmation multitâche et le fonctionnement d'un système d'exploitation temps réel embarqué.

Prise en main de FreeRTOS:

Avec Arduino:

Question 1 :
La différence entre les deux code BLINK et BLINKWITHOUTDELAY sur arduino:
Question 2 :
Nous avons connecté trois LEDs de couleurs différentes à notre arduino et avons tester ce programme

```
#define RED 6
#define YELLOW 7
#define BLUE 8



void setup() {
  // put your setup code here, to run once:
  pinMode(RED,OUTPUT);
  pinMode (YELLOW, OUTPUT);
  pinMode (BLUE, OUTPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(RED,digitalRead(RED)^1);
  digitalWrite(YELLOW,digitalRead(YELLOW)^1);
  digitalWrite(BLUE,digitalRead(BLUE)^1);
  delay (50);


}
```

Explication du code :

et comme résultat nous avons 3 LEDs qui clignotent toutes les unes après les autres si les tâche ont un ordre de priorité le scheduler commence par la tâche la plus prioritaire, s'il n' y a pas d'ordre de priorité il donne la priorité comme il le veut, dans ce cas il n' y a de scheduler donc l'ordre des priorité est selon comme c'est écrit ainsi le code effectue une tâche après l'autre.

Question 3 :

Pour un autre code on va inclure la bibliothèque <Arduino_FreeRTOS.h>

Dans ce code on utilise la fonction : **xTaskCreate** qui prend en argument : **redLedControllerTask** qui est une fonction de la tâche qu'on va effectuer, **"RED LED Task"** c'est le nom de la tâche, **128** le nombre de bit à allouer dans la mémoire pour effectuer cette tâche, **NULL: , 1** bit de priorité , **NULL**: optionnel et sert à agir sur la tâche.

redLedControllerTask : c'est une fonction qui gère l'état de la led, le paramètre void* pvParamaters est optionnel qui peut servir à transmettre les données à la tâche. Dans ce cas les LEDs clignotent les unes après les autres, il ya un scheduler qui définit l'ordre des priorité car elles ont toutes la même priorité.

Question 4 : profiling

On utilise toujours la bibliothéque <Arduino_FreeRTOS.h> ce qui veut dire qu'il y a un scheduler.

L'ordre des priorité est le même le scheduler définit donc lui même une priorité, mais si on modifie cet ordre on ne voit quand même pas de différence c'est-à-dire qu'on voit toutes les LEDs clignoter au même temps, ce qui explique ceci est la fonction delay qui crée un blocage dans chaque tâche mais ce

blocage est tellement court qu'on voit les LEDs avoir le même comportement.

Question 4 : TaskInstance

Dans ce cas nous avons utiliser le void *pvParametres qui est le paramètre de la fonction ledControllerTask, ce void *pvParametres est l'adresse qui contient l'état de la led et son pin ce qui permet d'avoir une seule fonction qui s'adapte à chaque led (comme le pin est en paramètre), cette fonction modifie l'état des leds. Une tâche dans ce cas représente la modification de l'état d'une des LEDs.

Il y a un schedular et dans le cas où nous avons définit la même priorité pour les tâches il v a les exécuter les unes après les autres, si on regarde le clignotement des leds il n 'est pas fait en même temps. Or si on définit un ordre de priorité le scheduler ne va exécuter que la tâche qui est la plus prioritaire.

Question5 : Task changing its priority

Dans ce cas nous avons utilisé en plus la fonction vTaskPrioritySet (NULL,2) dans la fonctionvoid yellowControllerTask(void *pvParameters), ce qu'elle fait est mettre la priorité l'ordre de priorité de la tâche à 2, dans ce cas c'est NULL car elle est appelé dans la tâche et donc c'est cette tâche qu'on met en priorité(on sait que les autres tâches ont un ordre de priorité de 1).

Si on supprime le delay on voit que seul la LED yellow clignote.

Question 6 : Getting Task Priority

Rien ne s'allume car on n'a pas de fonctions qui peuvent contrôler l'état de la led.

Ce programme utilise FreeRTOS pour gérer trois tâches qui exécutent des actions sur trois LED. Les LED rouge et bleue sont gérées par les tâches redControllerTask et blueControllerTask respectivement. La tâche yellowControllerTask gère la LED jaune et affiche la priorité de la tâche bleue à chaque itération.Les tâches ont différentes priorités, la tâche jaune ayant la priorité la plus élevée (3), et les tâches rouge et bleue ayant une priorité de 1.Chaque tâche affiche son "temps d'exécution" sur le moniteur série, mais il y a un petit problème : le delay(300) empêche une exécution multitâche réelle. Il serait plus efficace d'utiliser vTaskDelay()

Question 7 : Suspending task

Les 3 leds clignotent en même temps, après un certain temps la led yellow reste juste allumée et les deux autres continuent de clignoter.

Dans le code on voit qu'on a qu'au bout d'un certain nombre d'exécution de la tâche celle-ci est suspendu (c'est-à-dire l'état de la boucle ne change pas) .

On a modifié l'ordre des priorité et on voit que les 3 leds clignotent au même temps si on modifie la valeur du paramètre de v taskdelay on voit que les leds clignotent toute mais pas au même temps on peut déduire que v Taskdelay permet de **temporiser** une tâche sans bloquer tout le système, contrairement à la fonction delay qui bloque tout le système quand on définit un ordre de priorité.

Question 8 : Task suspending itself

On voit qu'on a une variable suspend_monitor qu'on incrémente dans la tâche void yellowLedControllerTask donc à chaque tâche effectuée par la led jaune cette variable s'incrémente et au bout de 7 incrémentation la led bleue s'éteint.

Et dans le moniteur série on voit que les temps d'exécution de chacune des leds est affiché, et visuellement on arrive à distinguer que ce temps est incrémenté après chaque clignotement de la led ce qu'on voit aussi dans le programme.

Question 9 : FreeRTOS - Reseuming a task

On voit que les leds clignotent en même temps puis la led rouge s'arrête ensuite elles reprennent mais pas avec les autres, et elle s'arrête au bout d'un certain nombre de clignotements et elles reprennent. Cette fois le clignotement est suspendu dans la tâche bleue c'est-à-dire au bout d'un certain nombre de clignotement de la led bleue, et le delay étant petit on voit les led clignoter très rapidement

Question 10 : Blocking a task with vTaskDelay

const TickType_t _500ms = pdMS_TO_TICKS(500); cette ligne convertit les millisecondes en ticks.

Dans le code on voit qu'on ne modifie pas les états des leds yellow et blue on affiche seulement sur le moniteur les messages suivants : This is BLUE , This is YELLOW.

Or la led red change d'état, donc elles clignotent.

Question 11 : using a queu

QueueCreate(5,sizeof(int32_t)) : Cette ligner permet de créer une file qui contient jusqu'à 5 éléments(entier de 32 bits).

Ensuite on crée les task envoyer et recevoir qui permettent l'envoi et transfert des données dans la file.

la tâche vSenderTask envoie un entier 1999 dans la file

la ligne qStatus =xQueueSend(yearQueue,&valueToSend,0); sert à envoyer la valeur 1999 et le paramètre 0 indique que la tâche ne va pas attendre si la file est pleine

vReceiverTask attend la réception de la donne envoye dans la file

si la file est vide , la tâche attend jusqu'à 100 ms puis essaie à nouveau

une fois la donnée est reçue , on la lit sur le moniteur série et le vTaskDelay(1) introduit un tick de delay

Question 12 : Queue sending from different tasks

cette fois , nous avons 2 tâches qui envoie respectivement les données 1999 et 2019 dans la file cree tearQueue*la fonction Receiver Task récupère les valeurs envoyées dans la file et les affiche sur le moniteur série

la led red est allumé pendant l'exécution de chaque tâche d'envoi et éteinte lorsqu'un message est reçu

Question 13 : Sending structures onto queues

dans ce code , on crée d'abord 2 structures

ensuite , on cree un tableau Data_t xStructToSend[2] qui continent deux structures à envoyer , une qui provient du sender 1 et une autre du sender 2

chacune des tâches envoie une des structures du tableau dans la file , et la tâche vReceiverTask reçoit les tâches dans la file et affiche la valeur envoyée et la source et les affiches sur le moniteur série

Si l'envoie échoue, un message d'erreur est affiché.

Question 14 : Mutex

Nous avons inclus la bibliothèque semphr.h qui fournit des fonctionnalités liées au sémaphores et aux mutex tel que :

Sémaphore : Outil de synchronisation qui permet à une ou plusieurs tâches d'attendre ou de signaler un événement. Il peut être **binaire** (verrouillage/déverrouillage) ou **comptage** (gérer plusieurs accès).

Mutex : Type spécial de sémaphore utilisé pour protéger une ressource partagée, empêchant plusieurs tâches de l'utiliser simultanément.

Ce code utilise FreeRTOS sur Arduino pour exécuter deux tâches concurrentes qui affichent des messages différents sur le moniteur série. Un mutex (xMutex) est utilisé pour éviter que les tâches n'écrivent simultanément, garantissant un affichage ordonné.

Nous avons ensuite préparé l'environnement STM32Cube IDE et avant testé les mêmes programmes avec les configurations nécessaires.

Description du système :

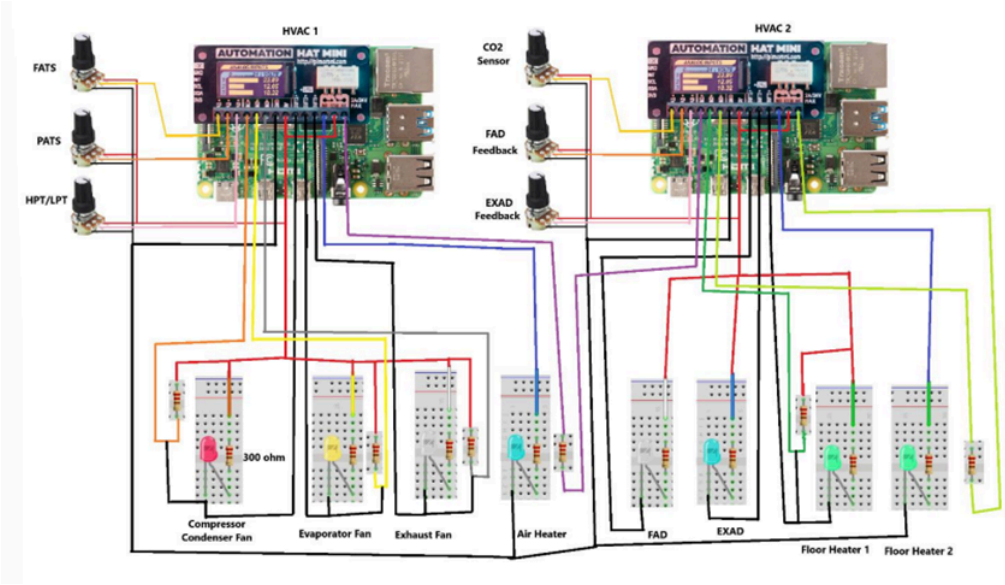


Figure 1 : schéma HVAC

Dans ce projet, nous sommes en charge du contrôleur HVAC2 qui gère une partie du système HVAC. ce nœud est responsable de :

- la **qualité de l'air** via le capteur de CO_2 et le **pilotage des dampers** FAD et EXAD.
- le **chauffage de l'air** à l'aide d'un Air Heater.
- le **chauffage au sol**, avec deux éléments chauffants simulés FH1 et FH2.

Les capteurs sont simulés par des potentiomètres, et les actionneurs (chauffages, dampers) par des LEDs (avec des résistances).

Cahier des charges :

Capteurs simulés (entrées analogiques) :

- CO_2 sensor : simulation du niveau de dioxyde de carbone dans l'air.

- FAD Feedback : retour de position du volet d'air frais.

- EXAD Feedback : retour de position du volet d'air d'extraction.

Actionneurs simulés (sorties logiques) :

- Air Heater : activation du chauffage d'air.
- FAD : ouverture variable selon le niveau de CO_2.
- EXAD : ouverture variable selon le niveau de CO_2.
- Floor Heater 1 : chauffage de sol local (zone 1)
- Floor Heater 2 : chauffage de sol local (zone 2)

Règles de régulations :

A) Régulation de la température (PATS)

- Si T ≤ 14 °C :

  ➢ Activer Air Heater, FH1, FH2

  ➢ FAD & EXAD fermés (0%), sauf si CO_2 > 1500 ppm

- Si 14 °C < T ≤ 17 °C :

  ➢ Activer Air Heater et FH1, désactiver FH2

  ➢ FAD & EXAD fermés, sauf si CO_2 > 1500 ppm

- Si 17 °C < T ≤ 19 °C :

  ➢ Activer Air Heater, désactiver FH1 et FH2

- Si T > 19 °C :

  ➢ Tous les chauffages éteints

B) Régulation du CO_2

- CO_2 < 600 ppm → FAD & EXAD fermés

- CO_2 > 700 ppm → FAD & EXAD à 25 %

- CO_2 > 1000 ppm → FAD & EXAD à 50 %

- CO_2 > 1500 ppm → FAD & EXAD à 75 %

- CO_2 > 2000 ppm → FAD & EXAD à 100 %

- CO_2 > 3000 ppm → alerte (à simuler ou afficher)
- CO_2 > 4000 ppm → défaut critique

Configuration du STM32 et implémentation des fonctions :

Pour la configuration sur STM32Cube IDE :

-> Pour commander les actionneurs simulés avec des **LEDs**
Par exemple :

- FH1 (LED 1)
- FH2 (LED 2)
- Air Heater (LED 3)

Ces pins doivent être en mode **GPIO Output .**

-> Pour lire les **potentiomètres** qui simulent les capteurs :

- PATS (température intérieure)
- CO_2 sensor
- FAD ou EXAD feedback (selon ton choix ou disponibilité)

Ces pins doivent être en **mode analogique**, reliés aux canaux de l'ADC.

Après la configuration nous nous sommes occupés du code :

Fonctionnalité	Utilisation
xTaskCreate()	création de deux tâches : Task_Sensors et Task_Control
vTaskDelay	vTaskDelay(pdMS_TO_TICKS(1000)) pour temporiser proprement
digitalWrite()	Remplacé par HAL_GPIO_WritePin(...) et HAL_GPIO_TogglePin(...)
analogRead()	Remplacé par Read_ADC_Channel(...) avec configuration dynamique de l'ADC
Serial.println()	Remplacé par printf(...)
Set_pwm()	Gère le PWM avec le timer TIM1, pour simuler l'ouverture des dampers
Variables globales	Déclaration de temperature, co2_ppm, fh1_state

Annexes :

Code :

```c
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "FreeRTOS.h"
#include "task.h"
#include <math.h>

/* USER CODE BEGIN Includes */
#include "core_cm4.h"  // Pour ITM
#include <stdio.h>
/* USER CODE END Includes */

/* Private variables ---------------------------------------------------------*/
ADC_HandleTypeDef hadc1;
TIM_HandleTypeDef htim1;

/* USER CODE BEGIN PV */
float temperature = 22.0f;
float co2_ppm = 0.0f;
uint8_t fad_pwm = 0, exad_pwm = 0;
uint8_t fh1_state = 0, fh2_state = 0;
/* USER CODE END PV */

/* USER CODE BEGIN PFP */
float Read_ADC_Channel(uint32_t channel);
void Set_PWM(TIM_HandleTypeDef *htim, uint32_t channel, uint8_t duty_cycle_percent);
void Read_CO2(void);
void Read_Temperature(void);
void Control_Damper(void);
void Control_Heater(void);
void Task_Sensors(void *argument);
void Task_Control(void *argument);
/* USER CODE END PFP */
```

```c
/* USER CODE BEGIN 0 */
int _write(int file, char *ptr, int len)
{
    for (int i = 0; i < len; i++) {
        ITM_SendChar(*ptr++);
    }
    return len;
}

float Read_ADC_Channel(uint32_t channel)
{
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = channel;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;

    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    float value = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);

    return value / 4095.0f;  // Normalisé [0,1]
}

void Set_PWM(TIM_HandleTypeDef *htim, uint32_t channel, uint8_t duty_cycle_percent)
{
    uint32_t pulse = (__HAL_TIM_GET_AUTORELOAD(htim) * duty_cycle_percent) / 100;
    __HAL_TIM_SET_COMPARE(htim, channel, pulse);
}

void Read_CO2()
```

```c
void Read_CO2()
{
    float analog = Read_ADC_Channel(ADC_CHANNEL_1); // PA1
    co2_ppm = analog * 2000.0f;
}

void Read_Temperature()
{
    float analog = Read_ADC_Channel(ADC_CHANNEL_2); // PA2
    temperature = analog * 40.0f;
}

void Control_Damper()
{
    uint8_t co2_cmd = 0, vent_cmd = 0;

    if(co2_ppm < 600) co2_cmd = 0;
    else if(co2_ppm <= 700) co2_cmd = 25;
    else if(co2_ppm <= 1000) co2_cmd = 50;
    else if(co2_ppm <= 1500) co2_cmd = 75;
    else co2_cmd = 100;

    if(temperature < 19) vent_cmd = 0;
    else if(temperature < 21) vent_cmd = 0;
    else if(temperature < 23) vent_cmd = 50;
    else if(temperature < 25) vent_cmd = 30;
    else if(temperature < 30) vent_cmd = 10;
    else vent_cmd = 5;

    fad_pwm = exad_pwm = fmin(co2_cmd + vent_cmd, 100);

    Set_PWM(&htim1, TIM_CHANNEL_1, fad_pwm); // PA7
    Set_PWM(&htim1, TIM_CHANNEL_2, exad_pwm); // PA4
```

```c
void Control_Heater()
{
    if (temperature < 14)
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET); // FH1 ON
        fh1_state = 1;

        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); // FH2 ON
        fh2_state = 1;
    }
    else
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET); // FH1 OFF
        fh1_state = 0;

        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); // FH2 TOGGLE
        fh2_state = !fh2_state;
    }
}

void Task_Sensors(void *argument)
{
    for(;;)
    {
        Read_CO2();
        Read_Temperature();

        // Affichage ITM
        printf("[Sensors] Temp: %.2f °C | CO2: %.2f ppm\r\n", temperature, co2_ppm);

        vTaskDelay(pdMS_TO_TICKS(1000)); // 1 seconde
    }
}
```

```c
void Task_Control(void *argument)
{
    for(;;)
    {
        Control_Damper();
        Control_Heater();

        // Affichage ITM
        printf("[Control] FAD_PWM: %d%% | EXAD_PWM: %d%% | FH1: %s | FH2: %s\r\n",
               fad_pwm, exad_pwm,
               fh1_state ? "ON" : "OFF",
               fh2_state ? "ON" : "OFF");

        vTaskDelay(pdMS_TO_TICKS(1000)); // 1 seconde
    }
}
/* USER CODE END 0 */

int main(void)
{
    /* MCU Configuration--------------------------------------------------------*/
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_TIM1_Init();

    /* USER CODE BEGIN 1 */
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1); // PA7
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2); // PA4
    /* USER CODE END 1 */
```

```
    /* USER CODE BEGIN 2 */
    xTaskCreate(Task_Sensors, "Sensors", 128, NULL, tskIDLE_PRIORITY + 1, NULL);
    xTaskCreate(Task_Control, "Control", 128, NULL, tskIDLE_PRIORITY + 1, NULL);
    vTaskStartScheduler();
    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        // Ne doit jamais s'exécuter
    }
    /* USER CODE END WHILE */
}

/* USER CODE BEGIN 4 */
// Tu peux ajouter ici d'autres fonctions utilisateur
/* USER CODE END 4 */
```